Supplementary Material for

ProbSPARQL: Querying Knowledge Graphs with Multi-dimensional, Uncertain Numeric Data

Table of Contents

S.1 Scope and Relationship to the Main Paper

The main paper presents ProbSPARQL as an upward-compatible query-layer extension for RDF knowledge graphs in which uncertain numeric measurements are represented by random-variable nodes linked to distribution-valued RDF literals. It introduces the random-variable modeling pattern, probabilistic literal datatypes, distribution-aware expressions, probabilistic filters, and divergence-based joins. The paper motivates these constructs in the circular-factory knowledge-graph infrastructure, using inspection-derived uncertain measurement data as the primary running example, and reports the principal evaluation results on project-derived measurement fragments and controlled ontology-conformant benchmarks.

This supplementary material provides supporting formal, implementation, and experimental details for ProbSPARQL. It specifies the probabilistic foundations, well-formedness constraints for probabilistic RDF literal datatypes, datatype-specific semantics for distribution-valued and deterministic-valued operators, evaluation procedures for scalar divergence functions and compatibility-oriented divergence matching, complete query workloads, benchmark construction procedures, extended evaluation protocols, algebraic properties of the ProbSPARQL evaluation model, and reproducibility details. It is intended as supporting material and does not introduce additional claims beyond those stated in the main paper.

S.2 Probabilistic Foundations

This section fixes the probabilistic notation used throughout the supplementary material. Following the main paper, ProbSPARQL abstracts away from the underlying sample space and represents an uncertain numeric measurement through its induced probability distribution. We therefore model such a measurement as a pair $X = (D, P)$, where $d \in \mathbb{N}^+$, $D \subseteq \mathbb{R}^d$ is a measurable value domain, $\mathcal{B}(D)$ denotes its Borel σ-algebra, and P is a probability measure on $(D, \mathcal{B}(D))$.

Distributional equivalence. Two random-variable representations $X = (D, P)$ and $X' = (D', P')$ are compatible if and only if $D = D'$. Compatible random-variable representations $X = (D, P)$ and $X' = (D, P')$ are distributionally equivalent, written $X \equiv X'$, if and only if $P = P'$ as probability measures. Equivalently, this means that $P(A) = P'(A)$ for every measurable set $A \in \mathcal{B}(D)$. This notion is semantic rather than lexical: two probabilistic RDF literals may be distributionally equivalent even if their lexical encodings are not identical.

Definition S.1 (Kullback–Leibler divergence). *Let P and Q be probability measures on $(D, \mathcal{B}(D))$. If P is absolutely continuous with respect to Q, written $P \ll Q$, the Kullback–Leibler divergence is*

$$\mathrm{KL}(P\|Q) = \int_D \log\left(\frac{dP}{dQ}\right) dP.$$

If $P \not\ll Q$, we set $\mathrm{KL}(P\|Q) = \infty$. When P and Q admit densities p and q with respect to a common reference measure λ, this becomes

$$\mathrm{KL}(P\|Q) = \int_D p(x) \log \frac{p(x)}{q(x)}\, d\lambda(x),$$

with the usual convention that $0 \log(0/q) = 0$ and that the divergence is infinite when $p > 0$ on a set where $q = 0$. KL divergence is non-negative, equals zero if and only if $P = Q$ as probability measures, and is generally asymmetric. We use natural logarithms throughout.

Definition S.2 (Jensen–Shannon divergence). *Let P and Q be probability distributions on the same measurable domain and let $M = \frac{1}{2}(P + Q)$. The Jensen–Shannon divergence is*

$$\mathrm{JSD}(P, Q) = \frac{1}{2}\mathrm{KL}(P\|M) + \frac{1}{2}\mathrm{KL}(Q\|M).$$

It is symmetric, satisfies $\mathrm{JSD}(P, Q) = 0$ if and only if $P = Q$ as probability measures, and is bounded by $0 \le \mathrm{JSD}(P, Q) \le \log 2$ when natural logarithms are used. Moreover, $\sqrt{\mathrm{JSD}}$ is a metric on probability distributions.

Definition S.3 (Divergence test). *Let $X = (D, P)$ and $X' = (D, P')$ be compatible random-variable representations, i.e., representations over the same measurable value domain, and let Δ be a divergence measure satisfying $\Delta(X, X') = 0$ if and only if $X \equiv X'$. Given a tolerance $\epsilon \ge 0$ and a decision parameter $\alpha \in (0, 1)$, the divergence test evaluates compatibility through*

$$H_0 : \Delta(X, X') \le \epsilon \qquad \textit{against} \qquad H_1 : \Delta(X, X') > \epsilon.$$

The test regards X and X' as compatible when H_0 cannot be rejected under the configured decision strategy.

In ProbSPARQL, the divergence test can be used either as a probabilistic filter function or as the dedicated `DIVJOIN` operator. The latter makes the divergence condition explicit in the query algebra. The parameter α has its standard significance-level interpretation when the configured backend provides a formal statistical error guarantee. For approximate or cascade-based decision strategies, it is treated as a decision-control parameter whose empirical error behavior is evaluated in the divergence-decision experiments.

Scalar divergence versus Boolean compatibility testing. ProbSPARQL distinguishes scalar-valued divergence functions from compatibility-oriented Boolean predicates. For compatible distribution-valued expressions, `prob:jsd`(E_1, E_2) returns a scalar divergence estimate and can be used in anomaly-detection filters, for example `prob:jsd`$(E_1, E_2) > 0.2$. In contrast, the divergence-test filter predicate with arguments $(E_1, E_2, \epsilon, \alpha)$ does not return a divergence value. It returns the Boolean compatibility decision defined above. This distinction separates score-based validation queries, such as anomaly detection, from compatibility-based matching queries, such as component pairing with `DIVJOIN`.

S.3 Probabilistic Literal Datatypes

ProbSPARQL represents uncertain numeric measurements by random-variable nodes that link a declared value domain to an encoded probability distribution. The distribution itself is stored as a probabilistic RDF literal, while the random-variable node provides the graph-level connection to the measured entity, declared value domain, unit, and measurement context. Each probabilistic datatype is defined by a lexical space, a value space, a partial interpretation function from lexical forms to datatype-specific distribution objects, and datatype-specific well-formedness constraints. Malformed lexical forms, inconsistent dimensionalities, unsupported encoding variants, or invalid parameter values cause the datatype interpretation to be undefined.

S.3.1 General Datatype Interpretation Scheme

For a probabilistic datatype τ, let L_τ be its lexical space and V_τ its value space. The datatype interpretation is a partial function

$$[\![\cdot]\!]_\tau : L_\tau \rightharpoonup V_\tau.$$

A probabilistic literal ℓ with datatype τ is well-formed if and only if its lexical form belongs to the domain of $[\![\cdot]\!]_\tau$. Operator evaluation is defined only for well-formed literals whose datatype, declared value domain, dimensionality, and operator-specific compatibility conditions are satisfied.

S.3.2 Gaussian Mixture Model Literals (`uq:gmmLiteral`)

A `uq:gmmLiteral` encodes a Gaussian mixture model with K components and Euclidean dimension d:

$$p(x) = \sum_{k=1}^{K} w_k \, \mathcal{N}(x; \mu_k, \Sigma_k),$$

where $x \in \mathbb{R}^d$, $w_k \geq 0$, $\sum_{k=1}^{K} w_k = 1$, $\mu_k \in \mathbb{R}^d$, and Σ_k represents a positive-definite covariance structure. The literal specifies a Gaussian mixture density on $\mathbb{R}^d$. Declared value-domain restrictions are not applied by the datatype interpretation itself.

Example and mathematical form. The following JSON lexical form encodes a two-component, two-dimensional GMM:

```
{
  "n_components": 2, "dimensions": 2, "covariance_type": "full",
  "weights": [0.3, 0.7], "means": [[2.5, 4.0], [6.0, 3.5]],
  "covariances": [
    [[1.2, 0.6], [0.6, 1.0]],
    [[0.8, -0.4],[-0.4, 1.5]]
  ]
}
```

Its mathematical form is

$$p(x) = 0.3\,\mathcal{N}\left(x; \begin{bmatrix}2.5\\4.0\end{bmatrix}, \begin{bmatrix}1.2 & 0.6\\0.6 & 1.0\end{bmatrix}\right) + 0.7\,\mathcal{N}\left(x; \begin{bmatrix}6.0\\3.5\end{bmatrix}, \begin{bmatrix}0.8 & -0.4\\-0.4 & 1.5\end{bmatrix}\right).$$

Key interpretation of the fields. The key `n_components` specifies the number K of Gaussian mixture components. The key `dimensions` specifies the Euclidean dimension d. The key `covariance_type` determines how each covariance structure is encoded. The key `weights` stores the mixture weights $w_1, \ldots, w_K$. The key `means` stores the component mean vectors $\mu_1, \ldots, \mu_K$. The key `covariances` stores the covariance structures $\Sigma_1, \ldots, \Sigma_K$ in the representation determined by `covariance_type`.

Field-level constraints. Let $K, d \in \mathbb{N}^+$ denote the number of components and the dimensionality of the GMM. A JSON lexical form is a well-formed `uq:gmmLiteral` if and only if it satisfies all of the following conditions:

- `n_components.` The field `n_components` is a positive integer K.
- `dimensions.` The field `dimensions` is a positive integer d.
- `covariance_type.` The field `covariance_type` is one of `"full"`, `"diag"`, or `"spherical"`.
- `weights.` The array `weights` has length K and encodes mixture weights $w_1, \ldots, w_K \in \mathbb{R}_{\geq 0}$ such that $\sum_{k=1}^{K} w_k = 1$ up to the configured numerical tolerance.
- `means.` The array `means` contains K arrays of length d, encoding mean vectors $\mu_1, \ldots, \mu_K \in \mathbb{R}^d$.
- `covariances.` The array `covariances` contains K entries encoding covariance structures $\Sigma_1, \ldots, \Sigma_K$, whose shapes are determined by the value of `covariance_type`:
  - `"full"`: each Σ_k is a symmetric positive-definite matrix in $\mathbb{R}^{d \times d}$.
  - `"diag"`: each Σ_k is a vector in $\mathbb{R}^d$ whose entries are strictly positive diagonal variances.
  - `"spherical"`: each Σ_k is a strictly positive scalar representing an isotropic variance.

S.3.3 Dirichlet Literals (`uq:dirichletLiteral`)

A `uq:dirichletLiteral` encodes a compositional random vector over the probability simplex

$$\Delta^{K-1} = \left\{ x \in \mathbb{R}^K \mid x_i \geq 0,\ \sum_{i=1}^{K} x_i = 1 \right\}.$$

It represents a random vector with K components, but its support is the $(K-1)$-dimensional simplex embedded in $\mathbb{R}^K$ rather than an unconstrained Euclidean domain.

Example and mathematical form. For example, the following JSON lexical form encodes a three-component Dirichlet distribution:

```
{"alphas": [5.0, 2.0, 3.0]}
```

It has concentration vector $\alpha = (5.0, 2.0, 3.0)$. The entries of `alphas` are concentration parameters, not a realization of the random vector. They must be strictly positive, but they are not required to sum to one. Samples drawn from the distribution lie on the simplex, i.e., $x_i \geq 0$ and $\sum_{i=1}^{K} x_i = 1$. Its density is

$$p(x) = \frac{1}{B(\alpha)} \prod_{i=1}^{3} x_i^{\alpha_i - 1}, \qquad x \in \Delta^2, \qquad B(\alpha) = \frac{\prod_{i=1}^{3} \Gamma(\alpha_i)}{\Gamma\left(\sum_{i=1}^{3} \alpha_i\right)}.$$

For this example, the mean composition is $\mathbb{E}[X] = \alpha/\alpha_0 = (0.5, 0.2, 0.3)$.

Key interpretation of the fields. The key `alphas` stores the concentration parameters $\alpha_1, \ldots, \alpha_K$. Its length determines the number of compositional components K. The normalized parameters determine the mean composition,

$$\mathbb{E}[X_i] = \frac{\alpha_i}{\alpha_0}, \qquad \alpha_0 = \sum_{j=1}^{K} \alpha_j.$$

For fixed normalized proportions, the total concentration α_0 controls how tightly the distribution is concentrated around its mean.

Field-level constraints. A JSON lexical form is a well-formed `uq:dirichletLiteral` if and only if it satisfies all of the following conditions:

- **Array length.** The field `alphas` is an array of length $K \geq 2$.
- **Parameter positivity.** Each concentration parameter satisfies $\alpha_i > 0$.

S.3.4 Histogram Literals (`uq:histogramLiteral`)

A `uq:histogramLiteral` provides a non-parametric representation for distributions over rectangular bin partitions. Its lexical form contains a `dimensions` field, a list of bin-boundary arrays, one for each dimension, and a tensor of bin masses. Histogram weights are interpreted as probability masses assigned to bins. For density evaluation, the represented distribution is treated as piecewise uniform inside each bin.

Example and mathematical form. The following JSON lexical form encodes a two-dimensional histogram:

```
{
  "dimensions": 2,
  "edges": [
    [0.0, 10.0, 20.0],
```

```
    [0.0, 5.0, 10.0]
  ],
  "weights": [
    [0.10, 0.20],
    [0.30, 0.40]
  ]
}
```

Let

$$B^{(1)} = (0, 10, 20), \qquad B^{(2)} = (0, 5, 10).$$

We use the convention that the tensor entry $w_{i_1,\ldots,i_d}$ is stored at the nested-array position $(i_1, \ldots, i_d)$. The example therefore defines four rectangular bins:

$$[0, 10) \times [0, 5), \quad [0, 10) \times [5, 10), \quad [10, 20) \times [0, 5), \quad [10, 20) \times [5, 10),$$

with probability masses 0.10, 0.20, 0.30, and 0.40, respectively. The density is piecewise uniform inside each bin. For example, in the first bin the density is

$$\frac{0.10}{(10-0)(5-0)} = \frac{0.10}{50}.$$

Key interpretation of the fields. The key `dimensions` specifies the number of dimensions d. The key `edges` stores one strictly increasing bin-boundary array for each dimension. The key `weights` stores the tensor of probability masses assigned to the corresponding bins. The tensor shape is determined by the number of bins along each dimension.

Field-level constraints. For dimensions $j = 1, \ldots, d$, let

$$B^{(j)} = \left(b_0^{(j)}, \ldots, b_{n_j}^{(j)}\right)$$

denote the bin-boundary array for dimension j. A JSON lexical form is a well-formed `uq:histogramLiteral` if and only if it satisfies all of the following conditions:

- **Dimension count.** The field `dimensions` is a positive integer d.
- **Bin boundaries.** The field `edges` contains exactly d arrays. Each array $B^{(j)}$ is strictly increasing.
- **Weight tensor.** The field `weights` is a non-negative tensor of shape $(n_1, \ldots, n_d)$.
- **Normalization.** The tensor entries sum to one up to the configured numerical tolerance.

The represented distribution is piecewise uniform over axis-aligned hyperrectangular bins. For a bin

$$C_{i_1,\ldots,i_d} = \prod_{j=1}^{d} \left[b_{i_j}^{(j)}, b_{i_j+1}^{(j)}\right),$$

with probability mass $w_{i_1,\ldots,i_d}$, the density inside the bin is

$$\frac{w_{i_1,\ldots,i_d}}{\prod_{j=1}^{d}\left(b_{i_j+1}^{(j)} - b_{i_j}^{(j)}\right)}.$$

The half-open interval convention avoids overlapping bins. The treatment of boundary points is immaterial for density evaluation because bin boundaries have measure zero.

S.4 Datatype-Specific Operator Semantics

This section specifies the semantics of the probabilistic expression operators used by ProbSPARQL and describes datatype-specific behavior for the probabilistic datatypes supported by the prototype. ProbSPARQL operators are partial. They are defined only when their operands are well-formed probabilistic literals or ordinary RDF literals of the expected type, and when datatype-specific dimensionality and compatibility requirements are satisfied. When an operator is undefined, expression evaluation yields an error, which is handled according to SPARQL's error propagation rules for expression and filter evaluation.

S.4.1 Expression Classes and Scope

ProbSPARQL distinguishes distribution-valued expressions from deterministic-valued expressions. Distribution-valued expressions return random-variable representations, whose distributions may be encoded as probabilistic RDF literals. Deterministic-valued expressions return ordinary RDF-compatible deterministic values, such as scalar numbers, vectors, Boolean values, or datatype-specific summaries. The expression forms considered in this supplement are:

- **Distribution-valued expressions:** scale, shift, linearTransform, marginal, joint, mix, multiply, fuse, and convolve.
- **Deterministic-valued expressions:** pdf, logpdf, cdf, mean, std, quantile, map, modeCount, and jsd.

These expressions are exposed through explicit probabilistic functions rather than by overloading SPARQL arithmetic symbols such as + and $*$. Arithmetic over scalar RDF literals therefore remains unchanged. Their semantics depend on the datatype backend and on the declared value domain, dimensionality, and compatibility conditions of the operands. The remaining subsections specify their result types, dimensionality requirements, compatibility conditions, and datatype-specific behavior.

S.4.2 Unary Distribution-Valued Transformations

This subsection specifies unary transformations from distribution-valued expressions to distribution-valued expressions. Let $X = (D, P)$ be a random-variable representation with $D \subseteq \mathbb{R}^d$.

Scale. scale(X, λ) returns the distribution of $Y = \lambda X$, where $\lambda \in \mathbb{R}$. For $\lambda \neq 0$, the value domain is $\lambda D = \{\lambda x \mid x \in D\}$, and the transformed density is $p_Y(y) = |\lambda|^{-d} p_X(y/\lambda)$. For a GMM, component means are scaled by λ, and covariance structures are scaled by λ^2, giving $\mu'_k = \lambda \mu_k$ and $\Sigma'_k = \lambda^2 \Sigma_k$. For histogram literals, scaling rescales bin boundaries, reorders them when necessary, and preserves the probability mass of each transformed bin when the result remains representable by the histogram backend. For $\lambda = 0$, the result is a degenerate point-mass distribution and is undefined unless the datatype backend supports point-mass distributions.

Shift. shift(X, v) returns the distribution of $Y = X + v$, where $v \in \mathbb{R}^d$. Its value domain is $D+v = \{x+v \mid x \in D\}$. For a GMM, this shifts every component mean by v and leaves mixture weights and covariance structures unchanged, giving $\mu'_k = \mu_k + v$ and $\Sigma'_k = \Sigma_k$. For histogram literals, shifting translates the bin-boundary arrays componentwise and preserves the probability mass of each bin. The operator is defined only when the shift vector has the same dimensionality as X.

Linear transformation. linearTransform(X, A, b) returns the distribution of $Y = AX + b$, where $A \in \mathbb{R}^{m \times d}$ and $b \in \mathbb{R}^m$. Its value domain is $AD+b = \{Ax+b \mid x \in D\}$, and the result is an m-dimensional distribution. For a GMM, each component is transformed as $\mu'_k = A\mu_k + b$ and $\Sigma'_k = A\Sigma_k A^\top$. If the transformation produces singular covariance matrices and the target datatype requires positive-definite covariances, the operator is undefined unless the backend supports degenerate Gaussian components. If a diagonal or spherical GMM becomes full-covariance after transformation, the backend serializes the result using a supported covariance representation. For histogram literals, general linear transformations are defined only when the backend can represent the transformed bins or remesh the transformed distribution. Otherwise, the operator is undefined.

Marginalization. marginal(X, S) returns the marginal distribution over a non-empty subset of dimensions $S \subseteq \{1, \ldots, d\}$. If π_S denotes the projection onto the dimensions in S, the resulting value domain is $\pi_S(D) = \{\pi_S(x) \mid x \in D\}$. For a GMM, marginalization selects the corresponding entries of each component mean and the corresponding covariance submatrix, giving $\mu'_k = \pi_S(\mu_k)$ and $\Sigma'_k = \Sigma_k[S, S]$. For histogram literals, marginalization sums probability masses over the dropped dimensions. For Dirichlet literals, selecting a subset of components does not in general yield another Dirichlet literal on a simplex, because the selected components need not sum to one. The operator is therefore defined only when the backend provides a datatype-specific representation for the marginal distribution or an explicit aggregation semantics. The operator is undefined when S is empty, when S contains dimensions outside $\{1, \ldots, d\}$, or when the target datatype cannot represent the result.

S.4.3 Binary Distribution-Valued Operators

Throughout this subsection, let $X_1 = (D_1, P_1)$ and $X_2 = (D_2, P_2)$ be random-variable representations. Binary operators that combine two random-variable representations at the value level require either an explicit joint distribution or an explicit independence assumption. When only marginal distributions are available, the joint behavior of the operands is underdetermined. The current prototype implements the independence-assumption case for operators whose backend explicitly declares this semantics.

Joint. $\mathrm{joint}(X_1, X_2)$ constructs a joint random-variable representation over the product domain $D_1 \times D_2$. In the current prototype, the expression is defined under the independence assumption declared by the backend and returns the product measure $P_1 \otimes P_2$. For independent GMM operands, the resulting mixture contains all pairwise component combinations. Its component weights are products of the original mixture weights, its means are concatenated component means, and its covariance matrices are block-diagonal:

$$w'_{k\ell} = w_k^{(1)} w_\ell^{(2)}, \qquad \mu'_{k\ell} = \begin{bmatrix} \mu_k^{(1)} \\ \mu_\ell^{(2)} \end{bmatrix}, \qquad \Sigma'_{k\ell} = \begin{bmatrix} \Sigma_k^{(1)} & 0 \\ 0 & \Sigma_\ell^{(2)} \end{bmatrix}.$$

Mixture. $\mathrm{mix}(X_1, X_2, \rho)$ returns the mixture distribution $\rho P_1 + (1-\rho)P_2$ for compatible domains and an outer mixture weight $\rho \in [0, 1]$. For GMM operands with compatible declared domains and the same Euclidean dimension, this concatenates the component sets and rescales their component weights by ρ and $1 - \rho$, respectively. The expression is undefined when the operands are not domain-compatible or when $\rho \notin [0, 1]$.

Convolution. $\mathrm{convolve}(X_1, X_2)$ returns the distribution of $Y = X_1 + X_2$ and is defined for operands over compatible Euclidean domains of the same dimensionality under the independence assumption declared by the backend. Its value domain is $D_1 + D_2 = \{x_1 + x_2 \mid x_1 \in D_1,\ x_2 \in D_2\}$. For densities p_1 and p_2, the density is

$$(p_1 * p_2)(y) = \int p_1(y - u) p_2(u)\, du.$$

For independent GMMs, convolution yields another GMM whose components are all pairwise sums of component means and covariances, with component weights given by products of the original mixture weights:

$$w'_{k\ell} = w_k^{(1)} w_\ell^{(2)}, \qquad \mu'_{k\ell} = \mu_k^{(1)} + \mu_\ell^{(2)}, \qquad \Sigma'_{k\ell} = \Sigma_k^{(1)} + \Sigma_\ell^{(2)}.$$

Multiplication. $\mathrm{multiply}(X_1, X_2)$ returns the distribution of $Y = X_1 X_2$ for scalar operands under the independence assumption declared by the backend. This is the form used in the motor-performance query, where uncertain power is derived from uncertain angular speed and torque. For vector-valued operands, the expression is defined only if the backend explicitly declares element-wise, inner-product, or

another product semantics. Since the exact product distribution is generally not closed under GMMs, the prototype may use sampling, moment matching, or another backend-specific approximation.

Fusion. $\mathrm{fuse}(X_1, X_2)$ combines two compatible distributions over a common domain D using a normalized product of densities:

$$p_{\mathrm{fuse}}(x) = \frac{p_1(x)p_2(x)}{\int_D p_1(u)p_2(u)\,du}.$$

The operator is defined only when the operands have a common measurable domain, admit densities with respect to a common reference measure, and the normalization constant is finite and non-zero. Fusion should not be confused with multiply. Fusion multiplies densities, whereas multiply multiplies random-variable values.

S.4.4 Deterministic-Valued Evaluation Operators

This subsection specifies evaluation operators that derive deterministic values from distribution-valued expressions. Their results include scalar numbers, vectors, probability values, divergence scores, and datatype-specific summaries. They do not return new random-variable representations.

Mean and standard deviation. $\mathrm{mean}(X)$ returns $\mathbb{E}[X]$. For a GMM, $\mathbb{E}[X] = \sum_{k=1}^{K} w_k\mu_k$. $\mathrm{std}(X)$ returns the standard deviation for scalar variables. For multivariate variables, it returns the component-wise standard deviation vector, i.e., the square roots of the diagonal entries of the distribution-level covariance matrix. For a GMM with mean $\mu = \sum_{k=1}^{K} w_k\mu_k$, this covariance matrix is

$$\mathrm{Cov}(X) = \sum_{k=1}^{K} w_k \left(\Sigma_k + (\mu_k - \mu)(\mu_k - \mu)^\top\right).$$

A richer covariance summary may be exposed through a separate datatype-specific operator.

PDF and log-PDF. $\mathrm{pdf}(X, t)$ and $\mathrm{logpdf}(X, t)$ evaluate the density or log-density at an evaluation point t of matching dimensionality. They are defined only for datatypes that provide a density interpretation. For histogram literals, density is evaluated according to the piecewise-uniform interpretation described in Section S.3.4. The density is zero outside the covered bin range, and log-density evaluation at zero-density points follows the backend's zero-density convention.

CDF. $\mathrm{cdf}(X, t)$ evaluates the cumulative distribution function. For multivariate operands, the evaluation point must be a vector $t = (t_1, \ldots, t_d)$ of matching dimensionality, and the CDF is interpreted as

$$F_X(t) = P(X_1 \le t_1, \ldots, X_d \le t_d).$$

Quantile. quantile(X, p) is defined for one-dimensional distributions $X = (D, P)$ with $D \subseteq \mathbb{R}$ and probability levels $p \in [0, 1]$. It returns a value according to the backend's quantile convention, typically $q = \inf\{x \in D \mid F_X(x) \geq p\}$. For continuous strictly increasing CDFs, this coincides with the usual inverse CDF. One-dimensional operators such as scalar quantiles are not applied directly to multivariate operands unless composed with marginal or linearTransform.

MAP and mode count. map(X) returns a maximum-density point under the datatype backend's density or mass interpretation, when such a point can be computed or approximated by the backend. For continuous densities, this corresponds to a point $x^\star \in \arg\max_{x \in D} p_X(x)$. When the distribution is interpreted as a posterior distribution, this point corresponds to the usual maximum-a-posteriori estimate. modeCount(X) returns the number of modes when this can be computed exactly or approximated by the backend. Both operators are backend-dependent for mixture and histogram representations. For histogram literals, the result depends on the backend-defined mode convention, such as counting modal bins or modal connected regions.

Jensen–Shannon divergence. jsd(X_1, X_2) returns a deterministic scalar divergence estimate for compatible distributions over a common measurable domain. Closed-form evaluation is generally unavailable for GMMs and other complex distribution families. The prototype therefore uses numerical or sampling-based approximation strategies. The scalar value returned by jsd is distinct from the Boolean compatibility decision returned by the divergence-test predicate or by `DIVJOIN`.

S.5 Divergence Functions and Divergence-Based Compatibility Decisions

This section describes how ProbSPARQL evaluates scalar divergence functions and Boolean compatibility decisions. The distinction follows the probabilistic foundations in Section S.2: `prob:jsd` returns a scalar divergence estimate, whereas the divergence-test filter predicate returns a Boolean compatibility decision. The `DIVJOIN` operator applies the same Boolean decision as a join condition.

S.5.1 Scalar Divergence Function

The deterministic-valued function `prob:jsd`(E_1, E_2) evaluates the Jensen–Shannon divergence between two compatible distribution-valued expressions over a common measurable domain. It is used when a query needs a scalar divergence value, for example in anomaly detection:

```
FILTER(prob:jsd(?d1, ?d2) > 0.2)
```

For distribution families without a closed-form JSD, the prototype uses numerical or sampling-based approximation. In the evaluation, these estimates are compared

against high-sample reference estimates to report mean absolute error (MAE). Thresholded scalar estimates are evaluated separately when they are used to induce binary decisions.

S.5.2 Boolean Compatibility Predicate

The divergence-test filter predicate, written with arguments $(E_1, E_2, \epsilon, \alpha)$, is the filter form of the divergence test defined in Definition S.3. It returns true when the configured decision strategy cannot reject the hypothesis that the divergence between the two operand distributions is at most ϵ. The `DIVJOIN` operator applies the same Boolean decision as an explicit join condition over designated distribution-valued variables. The interpretation of α follows the configured backend. The parameter α is a significance level only for backends with formal statistical error guarantees and otherwise acts as a decision-control parameter.

S.5.3 Decision Strategies

The prototype implements multiple strategies for evaluating the Boolean divergence decision introduced in Definition S.3. A strategy does not need to compute a high-precision scalar divergence for every candidate pair. It only needs to decide whether the null hypothesis $H_0 : \Delta(X, X') \leq \epsilon$ is rejected or not rejected under the configured decision procedure. Throughout this section, COMPATIBLE denotes non-rejection of H_0, not a proof that the true divergence is at most ϵ. The strategies differ in sampling budget, variance reduction, early-stopping behavior, and treatment of unresolved near-threshold cases.

- **Fixed-budget Monte Carlo.** Draws a predetermined number of samples, computes an estimate $\widehat{\Delta}_N$, and compares this estimate with the tolerance ϵ to obtain a binary decision.
- **Stratified Monte Carlo.** Uses the same fixed-budget decision rule, but allocates samples across mixture components or backend-defined strata. For Gaussian mixture operands, this exploits component structure to reduce estimator variance relative to unstratified sampling.
- **Sequential probability ratio test.** Applies Wald's sequential probability ratio test to evaluate the binary decision incrementally. The strategy may terminate before the maximum sampling budget when the accumulated likelihood ratio crosses one of the Wald decision boundaries.
- **Deterministic bound.** Computes a lower bound $\underline{\Delta}$ on the divergence. If $\underline{\Delta} > \epsilon$, the strategy rejects H_0 and returns INCOMPATIBLE. If $\underline{\Delta} \leq \epsilon$, the bound is inconclusive: the lower bound alone does not prove that $\Delta \leq \epsilon$. When used as a standalone strategy, inconclusive cases are treated as non-rejections of H_0 and returned as COMPATIBLE. When used inside the adaptive cascade, inconclusive cases are passed to the sampling-based stage.
- **Adaptive cascade.** Combines deterministic bounding, Wald-SPRT, and stratified Monte Carlo in increasing order of computational cost. It first applies the deterministic bound as an early rejection screen. For candidate

pairs not rejected by the bound, it applies the Wald-SPRT backend. If the sequential stage reaches its maximum sampling budget without crossing either Wald boundary, the cascade falls back to the stratified Monte Carlo estimate for the final binary decision.

Wald-SPRT backend. The prototype instantiates the sequential strategy as a Wald SPRT over batch-level evidence produced by the divergence estimator. Let $Y_1, \ldots, Y_n$ denote the evidence accumulated after n sampling batches. The backend uses two configured decision models with likelihoods f_0 and f_1, where f_0 corresponds to the compatible side of the tolerance boundary and f_1 corresponds to the incompatible side. The accumulated log-likelihood ratio is

$$L_n = \sum_{i=1}^{n} \log \frac{f_1(Y_i)}{f_0(Y_i)}.$$

For Wald-SPRT, the prototype uses the query-level parameter α as a symmetric nominal error setting, i.e., $\alpha_{\mathrm{I}} = \beta_{\mathrm{II}} = \alpha$. The decision boundaries are therefore

$$A = \log \frac{1-\alpha}{\alpha}, \qquad B = \log \frac{\alpha}{1-\alpha}.$$

The backend returns INCOMPATIBLE when $L_n \geq A$, returns COMPATIBLE when $L_n \leq B$, and continues sampling otherwise. If the maximum sampling budget $N_{\max}$ is reached without crossing either boundary, the standalone SPRT strategy compares the final estimate $\widehat{\Delta}_{N_{\max}}$ with ϵ. In the adaptive cascade, this unresolved case is handled by the stratified Monte Carlo fallback.

Deterministic-bound backend. For the JSD-based workloads considered in the evaluation, the deterministic-bound backend uses a lower bound obtained by measurable coarsening. For any measurable partition $\mathcal{A} = \{A_1, \ldots, A_M\}$ of the common measurable domain, let

$$\pi_{\mathcal{A}}(P) = (P(A_1), \ldots, P(A_M)) \quad \text{and} \quad \pi_{\mathcal{A}}(Q) = (Q(A_1), \ldots, Q(A_M))$$

be the induced discrete distributions. By the data-processing inequality,

$$\mathrm{JSD}(P, Q) \geq \mathrm{JSD}\big(\pi_{\mathcal{A}}(P), \pi_{\mathcal{A}}(Q)\big).$$

The implementation constructs an adaptive partition from the operand distributions and computes the discrete JSD on the induced bin masses. This discrete value is a lower bound, not a full divergence estimate. Therefore, a value above ϵ is sufficient for rejecting H_0, whereas a value at or below ϵ only leaves H_0 unresolved. As a standalone strategy, unresolved cases are treated as non-rejections of H_0. In the adaptive cascade, they are passed to the Wald-SPRT stage.

Adaptive cascade. The adaptive cascade first uses the deterministic-bound backend to reject candidate pairs whose lower bound already exceeds ϵ. Candidate pairs not rejected by the bound are evaluated by the Wald-SPRT backend. If the

sequential test terminates early, its decision is returned. If it exhausts the maximum sampling budget without crossing either Wald boundary, the cascade uses the stratified Monte Carlo estimate to make the final threshold decision against ϵ. The same stratified batches used for the sequential evidence also maintain the running estimate used by the final fallback decision. Algorithm 1 summarizes the procedure.

Algorithm 1 Adaptive-cascade divergence decision

Require: Distributions X, X', tolerance ϵ, decision parameter α
Require: Lower-bound function LB
Require: SPRT decision models f_0, f_1 over batch evidence
Require: Maximum sampling budget $N_{\max}$
Ensure: Binary decision COMPATIBLE or INCOMPATIBLE

$b \leftarrow \mathrm{LB}(X, X')$
if $b > \epsilon$ **then**
    return INCOMPATIBLE
end if
$L \leftarrow 0$
$A \leftarrow \log((1-\alpha)/\alpha)$
$B \leftarrow \log(\alpha/(1-\alpha))$
for $n = 1, \ldots, N_{\max}$ **do**
    Draw the next stratified sampling batch and obtain evidence Y_n
    Update the running stratified estimate $\widehat{\Delta}_n$
    $L \leftarrow L + \log(f_1(Y_n)/f_0(Y_n))$
    if $L \geq A$ **then**
        return INCOMPATIBLE
    else if $L \leq B$ **then**
        return COMPATIBLE
    end if
end for
if $\widehat{\Delta}_{N_{\max}} \leq \epsilon$ **then**
    return COMPATIBLE
else
    return INCOMPATIBLE
end if

S.6 Complete ProbSPARQL Query Workloads

This section gives complete query templates for the four motivating use cases. For readability, prefix declarations are omitted from individual listings. The queries use the same prefixes as the main paper, including `ag:` for the angle-grinder vocabulary, `im:` for inspection measurements, `om:` for units of measurement, and `uq:` for probabilistic random-variable and literal vocabulary.

S.6.1 R1: Threshold-Based Retrieval

```
SELECT ?gear ?lenchar ?prob WHERE {
  ?gear a ag:CrownGear ;
        im:hasLengthCharacteristic ?lenchar .
  ?lenchar im:isMeasuredBy ?measurement .
  ?measurement om:hasUnit om:millimetre ;
               uq:representedBy ?rv .
  ?rv uq:hasDomain uq:nonNegativeDouble ;
      uq:hasDistribution ?dist .
  BIND(prob:cdf(?dist, "9.8"^^xsd:double) AS ?prob)
  FILTER(?prob >= "0.9"^^xsd:double)
}
```

Listing S.1. R1: Threshold-based retrieval using a probabilistic CDF filter.

Listing S.1 returns each crown gear together with a tooth-length characteristic whose distribution assigns probability at least 0.9 to lengths below the 9.8 mm threshold. Thus, a gear is retrieved whenever at least one associated tooth-length characteristic satisfies the probabilistic threshold.

S.6.2 R2: Anomaly Detection with Scalar Divergence

```
SELECT ?gear ?char ?div WHERE {
  ?gear a ag:CrownGear ;
        im:hasLengthCharacteristic ?char .
  ?char im:isMeasuredBy ?ctMeasurement ;
        im:isMeasuredBy ?slMeasurement .
  ?ctMeasurement a im:ComputedTomographyMeasurement ;
                 om:hasUnit om:millimetre ;
                 uq:representedBy ?rvCT .
  ?slMeasurement a im:StructuredLightMeasurement ;
                 om:hasUnit om:millimetre ;
                 uq:representedBy ?rvSL .
  ?rvCT uq:hasDomain ?domain ;
        uq:hasDistribution ?distCT .
  ?rvSL uq:hasDomain ?domain ;
        uq:hasDistribution ?distSL .
  BIND(prob:jsd(?distCT, ?distSL) AS ?div)
  FILTER(?div > "0.2"^^xsd:double)
}
```

Listing S.2. R2: Anomaly detection using scalar Jensen–Shannon divergence.

Listing S.2 compares CT and structured-light distributions for the same tooth-length characteristic. The shared `?domain` binding and the common millimetre unit ensure that `prob:jsd` is evaluated only on compatible length distributions. The query returns the gear, the characteristic that triggered the anomaly, and the scalar JSD value.

S.6.3 R3: Motor Performance Evaluation by Uncertainty Propagation

```
SELECT ?motor ?powerDist ?powerUnit WHERE {
  ?motor a ag:Motor ;
         ag:hasSpeedMeasurement ?speedMeasurement ;
         ag:hasTorqueMeasurement ?torqueMeasurement .
  ?speedMeasurement om:hasUnit om:radianPerSecond ;
                    uq:representedBy ?speedRV .
  ?torqueMeasurement om:hasUnit om:newtonMetre ;
                     uq:representedBy ?torqueRV .
  ?speedRV uq:hasDomain uq:nonNegativeDouble ;
           uq:hasDistribution ?speedDist .
  ?torqueRV uq:hasDomain uq:nonNegativeDouble ;
            uq:hasDistribution ?torqueDist .
  BIND(prob:multiply(?speedDist, ?torqueDist) AS ?powerDist)
  BIND(om:watt AS ?powerUnit)
}
```

Listing S.3. R3: Uncertainty propagation from angular speed and torque to power.

Listing S.3 computes a distribution over mechanical power by multiplying the angular-speed and torque distributions. The query assumes that both operands are scalar random variables expressed in the canonical units required for $P = \omega\tau$, here radians per second and newton metres. In this template, `prob:multiply` treats the two operands as independent unless a joint distribution or a datatype-specific dependency model is provided.

S.6.4 R4: Component Pairing with Divergence Join

```
SELECT ?driveGear ?spindleGear WHERE {
  {
    ?driveGear a ag:DriveGear ;
      im:hasHardCharacteristic ?hardcharA .
    ?hardcharA im:isMeasuredBy ?measA .
    ?measA om:hasUnit om:HardnessVickers ;
      uq:representedBy ?rvA .
    ?rvA uq:hasDomain uq:nonNegativeDouble ;
      uq:hasDistribution ?distA .
  }
  DIVJOIN(?distA, ?distB, 0.3, 0.05)
  {
    ?spindleGear a ag:SpindleGear ;
      im:hasHardCharacteristic ?hardcharB .
    ?hardcharB im:isMeasuredBy ?measB .
    ?measB om:hasUnit om:HardnessVickers ;
      uq:representedBy ?rvB .
    ?rvB uq:hasDomain uq:nonNegativeDouble ;
      uq:hasDistribution ?distB .
  }
}
```

Listing S.4. R4: Divergence join for compatibility-oriented component pairing.

Listing S.4 is the supplementary version of the R4 query shown in the main paper. It retrieves candidate drive–spindle gear pairs, follows their hardness-measurement links to the corresponding random-variable nodes, and compares

the associated hardness distributions with `DIVJOIN`. The join returns pairs for which the null hypothesis $H_0 : \Delta \leq \epsilon$ cannot be rejected under the configured decision strategy, using $\epsilon = 0.3$ and $\alpha = 0.05$. Matching is therefore based on distributional compatibility rather than scalar equality.

S.7 Benchmark Construction

S.7.1 Generated Circular-Factory Knowledge Graph

The benchmark graphs mirror the circular-factory schema used in the main paper, including angle-grinder instances, component instances, measurable characteristics, measurement records, random-variable nodes, unit annotations, scalar summaries, and probabilistic RDF literals. The benchmark family is constructed for controlled experiments and ranges from 10 angle-grinder instances with 3,078 triples to 5,000 angle-grinder instances with 1,535,008 triples. Different experiments use different members of this benchmark family: distribution-complexity experiments fix the graph size, whereas graph-size scalability experiments vary the number of generated angle-grinder instances. Table S.1 summarizes the benchmark configurations used for these controlled experiments.

The generated graphs preserve the graph traversal structure of the circular-factory query layer while making distribution parameters, filter selectivity, and divergence difficulty independently controllable. They are therefore used as reproducible workloads for controlled experiments, rather than as a claim to reproduce the full empirical SFB 1574 knowledge graph.

Table S.1. Size and composition of the controlled circular-factory benchmark graphs. P = products, C = components, Char. = measurable characteristics, Meas. = measurement records, RVs = random-variable nodes.

Graph	P	C	Char.	Meas.	RVs	Triples
S1	10	30	80	240	240	3,078
S2	100	300	800	2,400	2,400	30,708
S3	1,000	3,000	8,000	24,000	24,000	307,008
S4	5,000	15,000	40,000	120,000	120,000	1,535,008

S.7.2 Distribution Generation

For scalar inspection quantities modelled by one-dimensional GMMs, the number of mixture components is varied over $K \in \{1, 3, 5, 10\}$. Mixture weights are sampled as positive values and normalized to sum to one. Component means are drawn from numeric ranges corresponding to the inspection quantities used in the query workloads, and variance parameters are sampled from strictly positive

intervals to ensure valid Gaussian components and avoid degenerate zero-variance distributions. Random seeds are fixed in the benchmark scripts to make graph generation and query workload construction reproducible.

The generated distributions support controlled selectivity for threshold filters and controlled decision difficulty for divergence workloads. For threshold-filter workloads, selectivity is controlled at workload-construction time by selecting query thresholds that produce target pass fractions on the generated graph. For divergence workloads, a pool of distribution pairs is generated and later grouped according to reference JSD-based decision difficulty, as specified in Section S.7.3.

Distribution-complexity experiments vary K while keeping the graph size fixed. Graph-size scalability experiments fix $K = 3$ and vary the number of generated angle-grinder instances. This separation avoids conflating the cost of probabilistic literal evaluation with the cost of RDF graph matching over larger benchmark graphs.

S.7.3 Divergence-Decision Workloads

Candidate pairs for divergence-decision evaluation are grouped by decision difficulty. For each candidate pair i, we estimate a high-precision reference Jensen–Shannon divergence (JSD) value Δ_i^{ref} using 10^6 Monte Carlo samples. These reference estimates are used only for workload construction and evaluation, not during ordinary query execution. Unless otherwise stated, the compatibility tolerance is set to $\epsilon = 0.3$. We define the decision margin of pair i as

$$m_i = \left| \Delta_i^{\mathrm{ref}} - \epsilon \right|.$$

Small margins correspond to near-threshold pairs whose compatibility decision is sensitive to estimation error, whereas large margins correspond to pairs that are clearly below or above the tolerance threshold.

The difficulty regimes are constructed from these margins. Easy pairs satisfy $m_i \geq 0.15$, Medium pairs satisfy $0.05 \leq m_i < 0.15$, and Hard pairs satisfy $m_i < 0.05$. Thus, the Hard workload consists of near-threshold pairs whose reference JSD lies within $\epsilon \pm 0.05$, matching the definition used in the main paper. The Mixed workload is constructed by sampling 800 candidate pairs from each of the Easy, Medium, and Hard regimes, giving $N = 2400$ candidate pairs in total.

Each Easy, Medium, and Hard workload contains $N = 2400$ candidate pairs sampled from the fixed-size S3 benchmark graph with 1,000 generated angle-grinder instances. The resulting workloads are used to compare divergence-decision strategies, including fixed-budget Monte Carlo, stratified Monte Carlo, sequential probability ratio testing (SPRT), a deterministic lower-bound strategy, and Adaptive-Cascade. Because the graph size and candidate-pair count are fixed, these measurements isolate the per-candidate-pair cost and accuracy of divergence decisions from graph-size scalability effects.

S.7.4 Execution Environment

All Java-based experiments are run on the same hardware and software stack: an Apple MacBook Pro with an Apple M4 Pro chip, 48 GB RAM, macOS 15.7.4,

Java 21, and Apache Jena 6.0.0. Queries are executed through the extended Jena ARQ engine. When endpoint execution is evaluated, the same engine is exposed through Apache Jena Fuseki. The exact Maven dependency versions and build configuration are pinned in the released artifact. Unless otherwise stated, each reported latency is the median of 10 runs after 3 warm-up iterations. Warm-up runs are excluded from the reported measurements. Application-layer post-processing baselines are executed on the same machine.

S.8 Extended Evaluation Protocols and Result Tables

This section provides extended protocols and result tables supporting the evaluation in the main paper. It first reports the real-data applicability check on project-derived circular-factory measurement fragments used to support EQ1. It then gives the controlled benchmark protocols and results for distribution-complexity overhead, knowledge-graph size scalability, filter-pushdown behavior, divergence-decision strategies, end-to-end divergence-join runtime, dimensionality scaling for GMM literals, and datatype extensibility.

Graph-size scaling and distribution-complexity scaling are evaluated separately. Graph-size scaling varies the number of ontology-conformant angle-grinder instances, while distribution-complexity scaling fixes the benchmark size and varies the number of mixture components K. The protocols below specify how reference estimates, accuracy metrics, benchmark configurations, and latency measurements are obtained.

Unless otherwise stated, latency values are reported as medians over repeated runs after warm-up, following the evaluation protocol in Section S.7.4.

S.8.1 Applicability on Real Circular-Factory Measurement Data

Before evaluating runtime behavior on controlled benchmark graphs, we validate ProbSPARQL on a circular-factory knowledge-graph fragment derived from project measurement data. This applicability check examines whether the proposed modelling pattern can represent and query real uncertain measurements, rather than only generated distributions.

The fragment contains 4,037 RDF triples across three project-derived measurement subsets. It includes 97 sample resources, 20 process-condition resources, 280 measurement resources, and 163 random-variable resources. The data describe material-analysis measurements for drive and spindle gears, including Barkhausen-noise root-mean-square signal values, residual-stress measurements, hardness measurements, and associated process conditions. Selected Barkhausen-noise and residual-stress measurements are represented as random variables with Gaussian-mixture literals. A separate roughness subset uses histogram literals to represent empirical distributions.

The real-data fragment uses the same modelling pattern as the controlled benchmarks: samples are linked to measured characteristics, measurements, random-variable nodes, physical units, and probabilistic distribution literals.

Therefore, the query templates in Section S.6 can be instantiated on project measurement data without changing the random-variable access pattern or the probabilistic operator structure.

The representative real-fragment tasks are as follows.

- **RMS threshold retrieval for material-analysis samples.** Query: Which gear samples have a Barkhausen-noise RMS distribution whose upper-tail probability exceeds the operational threshold, e.g., $P(\mathrm{RMS} > 700) \geq 0.9$, under the recorded process condition? Capability: CDF-based upper-tail filtering. Source: R1 pattern adapted to the RMS measurement subset and executable artifact query.
- **Consistency checking for repeated material-analysis measurements.** Query: Which repeated Barkhausen-noise or residual-stress measurements of the same sample under the same process condition show a distributional deviation above the configured divergence threshold? Capability: Scalar divergence evaluation. Source: R2 pattern adapted to repeated project measurements and executable artifact query.
- **Hardness-based compatibility search for gear pairing.** Query: Which drive–spindle gear candidates have compatible hardness distributions, i.e., pairs for which the divergence test does not reject $H_0 : \Delta \leq \epsilon$ under the configured decision strategy? Capability: Divergence join. Source: R4 pattern adapted to the hardness measurement subset and executable artifact query.
- **CDF evaluation for empirical roughness distributions.** Query: For roughness measurements represented by histogram literals, which cumulative probabilities are obtained at the engineering-relevant roughness thresholds used in the project fragment? Capability: Histogram-literal CDF evaluation. Source: Executable artifact query for the roughness subset.

The executable real-fragment queries instantiate the reusable patterns in Section S.6 with project-specific measurement predicates, units, thresholds, and datatype literals. References to R1, R2, and R4 therefore identify the corresponding query patterns, not necessarily verbatim queries over the project-derived fragment. The histogram query is included in the artifact package because it is specific to the roughness subset and does not correspond one-to-one to the four main use-case templates. The purpose of this applicability check is not to report scalability results. Rather, it checks that project-derived measurements can be represented as random variables, retrieved through ordinary RDF graph patterns, and consumed by ProbSPARQL operators. Runtime and scalability effects are evaluated separately on controlled ontology-conformant benchmarks.

S.8.2 Reference Divergence Estimates

For divergence-evaluation experiments, reference JSD values are estimated with 10^6 Monte Carlo samples per candidate pair. These values are used as empirical references only for workload construction and evaluation.

Table S.2. Median latency under different GMM mixture-component counts. The K-columns report ProbSPARQL GMM latency in milliseconds. Det. denotes the corresponding deterministic, non-probabilistic query variant. Distribution comparison has no deterministic counterpart.

Query type	**Det.**	**ProbSPARQL (ms)**				**Overhead**
	(ms)	$K = 1$	$K = 3$	$K = 5$	$K = 10$	(×)
Retrieval	170.66	242.43	213.44	233.45	220.17	1.25–1.42
Probabilistic filtering	172.16	275.31	245.66	249.43	248.21	1.43–1.60
Uncertainty propagation	31.91	55.21	54.31	57.81	62.13	1.70–1.95
Distribution comparison	–	1852.80	4339.60	6490.01	10744.45	–

For numerical divergence estimates, the reference values are used to compute mean absolute error (MAE):

$$\mathrm{MAE} = \frac{1}{N} \sum_{i=1}^{N} \left| \widehat{\Delta}_i - \Delta_i^{\mathrm{ref}} \right|,$$

where N is the number of evaluated candidate pairs, $\widehat{\Delta}_i$ is the divergence estimate produced by the evaluated strategy, and Δ_i^{ref} is the corresponding reference estimate.

For binary divergence decisions, reference JSD values are converted into reference labels using the tolerance $\epsilon = 0.3$. A pair is labelled reference-compatible if $\Delta_i^{\mathrm{ref}} \leq \epsilon$ and reference-incompatible otherwise. The reference-compatible class is used as the positive class. Predicted decisions are then compared with these reference labels to compute precision, recall, and F1 score. MAE evaluates numerical divergence estimation, whereas precision, recall, and F1 evaluate the induced binary divergence decision.

S.8.3 Distribution-Complexity Overhead

This experiment isolates the effect of distributional complexity by varying the number of GMM components K on a fixed-size benchmark graph. It is therefore separate from the graph-size scalability experiment, which varies the number of generated angle-grinder instances. For each query type, the ProbSPARQL variant is compared with its corresponding deterministic counterpart where such a counterpart exists. The deterministic counterparts use scalar summaries and ordinary SPARQL expressions while preserving the same graph-pattern structure.

Table S.2 reports the resulting median latencies and overhead ranges for retrieval, probabilistic filtering, uncertainty propagation, and distribution comparison.

As shown in Table S.2, retrieval, probabilistic filtering, and uncertainty propagation incur moderate overhead across $K \in \{1, 3, 5, 10\}$. The weak and non-monotonic dependence on K indicates that K is not the primary cost driver for

Table S.3. Knowledge-graph size scalability with fixed GMM mixture-component count $K = 3$. Graph sizes are defined in Table S.1. Latencies are median execution times in milliseconds.

Graph	Det. Ret.	Prob. Ret.	Det. Filt.	Prob. Filt.	Prob. Comp.
S1	0.89	0.68	0.62	0.81	21.52
S2	0.79	1.30	1.11	1.52	191.20
S3	15.67	42.64	16.71	43.12	2,412.23
S4	170.66	213.44	172.16	245.66	4,339.60

these workloads at this benchmark size. This is consistent with fixed graph-pattern evaluation and probabilistic literal handling contributing a substantial part of the runtime. Distribution comparison behaves differently: its latency increases substantially with K, reflecting the higher cost of sampling-based divergence estimation.

S.8.4 Knowledge-Graph Size Scalability

Graph-size scalability is evaluated on graphs S1–S4 from Table S.1, varying the number of generated angle-grinder instances from 10 to 5,000 while fixing $K = 3$. Table S.3 reports median latencies for representative deterministic and ProbSPARQL query variants. Det., Prob., Ret., Filt., and Comp. denote deterministic, ProbSPARQL, retrieval, filtering, and distribution comparison, respectively.

As shown in Table S.3, the scalability experiment characterizes growth trends with respect to RDF graph size rather than the effect of distributional complexity. Retrieval and filtering show broadly similar qualitative growth patterns for the deterministic and ProbSPARQL variants, especially on the larger benchmark graphs. At the smallest graph sizes, sub-millisecond differences should not be interpreted as systematic speed differences. Distribution comparison incurs a substantially higher absolute cost. Its scaling also depends on the number of evaluated distribution pairs rather than only on the RDF graph size.

S.8.5 Filter Pushdown and Post-Processing Baseline

The filter-pushdown experiment compares in-engine probabilistic filtering against application-layer post-processing. The in-engine strategy evaluates the CDF predicate inside the SPARQL engine before materializing and transferring the filtered results. The post-processing baseline retrieves all relevant distribution literals and applies the same CDF predicate in the application layer. Speedup is computed as post-processing latency divided by in-engine latency. Table S.4 reports the resulting median latencies and speedups.

As shown in Table S.4, the post-processing baseline remains nearly constant across pass fractions because all candidate distribution literals must be materialized before filtering. In-engine evaluation applies the predicate before result

Table S.4. Median CDF-filter latency at different pass fractions under in-engine evaluation and application-layer post-processing. Speedup is computed as post-processing latency divided by in-engine latency.

Pass fraction	In-engine (ms)	Post-processing (ms)	Speedup
0.01	521.65	1779.20	3.41×
0.05	535.29	1856.25	3.47×
0.10	610.96	1791.18	2.93×
0.30	938.36	1842.93	1.96×

transfer, so it benefits most at low pass fractions. The observed speedups range from 1.96× to 3.47×.

S.8.6 Divergence-Decision Strategy Results

Divergence joins require binary decisions over candidate pairs. This experiment isolates the per-candidate-pair cost and accuracy of alternative divergence-decision strategies. Each workload contains $N = 2400$ candidate pairs sampled from the fixed-size S3 benchmark graph with 1,000 generated angle-grinder instances and uses $\epsilon = 0.3$ as the tolerance. The goal is not to identify a universally optimal strategy, but to characterize latency–accuracy trade-offs.

As shown in Table S.5, no decision strategy dominates across all metrics. MC-Stratified achieves the highest F1 score on the hard workload, but its latency remains comparable to MC-Naive. Compared with the fixed-budget Monte Carlo variants, SPRT and Adaptive-Cascade substantially reduce latency, but their recall decreases on the hard near-threshold workload. Det-Bound is orders of magnitude faster because it evaluates a conservative lower bound rather than a full JSD estimator. For Det-Bound, MAE quantifies the deviation of the bound from the reference JSD and should not be interpreted as the error of a full divergence estimator. Its high recall and lower precision reflect a bias toward classifying pairs as reference-compatible. These results motivate exposing the divergence-decision strategy as an application-dependent configuration rather than hard-coding a single default.

S.8.7 End-to-End Divergence-Join Runtime

Table S.5 reports per-candidate-pair decision costs and accuracy. We next evaluate the same strategies inside the complete `DIVJOIN` query on the same Easy, Medium, Hard, and Mixed workloads. Runtime is measured at the level of complete query execution and therefore includes graph-pattern evaluation, candidate-pair enumeration, divergence decisions, and result materialization. Table S.6 reports the resulting complete-query runtimes.

The Easy and Medium workloads each contain 150 reference-compatible pairs, i.e., pairs whose reference JSD does not exceed $\epsilon = 0.3$. Since all strategies achieve

Table S.5. Precision, recall, F1, MAE, and median per-candidate-pair latency of divergence-decision strategies across workload difficulties. Precision, recall, and F1 are reported as percentages. Each workload contains $N = 2400$ candidate pairs. Pairs with reference JSD not exceeding $\epsilon = 0.3$ are treated as the positive class.

Workload	Method	Prec. (%)	Rec. (%)	F1 (%)	MAE	Lat. (ms)
Easy	MC-Naive	100.00	100.00	100.00	0.000932	24.602
Easy	MC-Stratified	100.00	100.00	100.00	0.000523	23.991
Easy	SPRT	100.00	100.00	100.00	0.002203	0.260
Easy	Det-Bound	100.00	100.00	100.00	0.003713	0.009
Easy	Adaptive-Cascade	100.00	100.00	100.00	0.002681	0.124
Medium	MC-Naive	100.00	100.00	100.00	0.003717	23.927
Medium	MC-Stratified	100.00	100.00	100.00	0.001985	23.940
Medium	SPRT	100.00	100.00	100.00	0.006310	0.376
Medium	Det-Bound	100.00	100.00	100.00	0.012443	0.005
Medium	Adaptive-Cascade	100.00	100.00	100.00	0.010274	0.195
Hard	MC-Naive	96.05	97.33	96.69	0.004103	25.683
Hard	MC-Stratified	96.73	98.67	97.69	0.002125	23.337
Hard	SPRT	95.24	93.33	94.28	0.006199	4.468
Hard	Det-Bound	90.36	100.00	94.94	0.018363	0.005
Hard	Adaptive-Cascade	98.59	93.33	95.89	0.010707	3.269
Mixed	MC-Naive	100.00	98.64	99.32	0.002770	25.089
Mixed	MC-Stratified	100.00	99.32	99.66	0.001498	25.887
Mixed	SPRT	97.96	97.96	97.96	0.004767	1.810
Mixed	Det-Bound	96.69	99.32	97.99	0.011481	0.012
Mixed	Adaptive-Cascade	99.32	98.64	98.98	0.007800	1.039

perfect precision and recall on these far-from-threshold workloads in Table S.5, they all return the same 150 compatible pairs. Differences in returned-pair counts appear on the Hard and Mixed workloads, where near-threshold cases make the binary decision more sensitive to estimation and stopping behavior.

Table S.6. End-to-end runtime of the complete `DIVJOIN` query under different divergence-decision strategies across workload difficulties. Each workload contains $N = 2400$ candidate pairs. Returned pairs denotes the number of candidate pairs accepted as compatible by the strategy. Latencies are measured as median complete-query runtime in milliseconds. Speedup is computed relative to MC-Naive within the same workload.

Workload	Strategy	Returned pairs	Latency (ms)	Speedup
Easy	MC-Naive	150	63284.73	1.00
Easy	MC-Stratified	150	60917.46	1.04
Easy	SPRT	150	3186.42	19.86
Easy	Det-Bound	150	2294.87	27.58
Easy	Adaptive-Cascade	150	2678.35	23.63
Medium	MC-Naive	150	61472.58	1.00
Medium	MC-Stratified	150	61638.91	1.00
Medium	SPRT	150	3617.74	16.99
Medium	Det-Bound	150	2368.52	25.96
Medium	Adaptive-Cascade	150	3094.66	19.86
Hard	MC-Naive	152	66908.37	1.00
Hard	MC-Stratified	153	59876.12	1.12
Hard	SPRT	147	14762.83	4.53
Hard	Det-Bound	166	2826.41	23.67
Hard	Adaptive-Cascade	142	11874.29	5.64
Mixed	MC-Naive	145	64735.26	1.00
Mixed	MC-Stratified	146	66392.48	0.98
Mixed	SPRT	147	7826.57	8.27
Mixed	Det-Bound	151	2638.94	24.53
Mixed	Adaptive-Cascade	146	5764.31	11.23

Table S.6 shows that fixed-budget Monte Carlo strategies dominate complete-query runtime, requiring roughly 60–67 seconds across the workload regimes. SPRT and Adaptive-Cascade reduce runtime through early stopping, with the largest speedups on Easy and Medium workloads and smaller gains on the Hard workload. This pattern reflects the higher cost of near-threshold decisions, where more samples are needed before a stable compatibility decision can be made.

Det-Bound gives the lowest runtime in all workloads, but its returned-pair counts differ most visibly on the Hard workload. It should therefore be read together with the accuracy results in Table S.5. Adaptive-Cascade offers the more balanced query-level trade-off: it is consistently faster than SPRT and much faster

Table S.7. Median latency for GMM literals with fixed $K = 3$ and varying dimension d. Latencies are reported in milliseconds. Det. denotes the corresponding deterministic query variant. The overhead range is computed relative to the deterministic baseline.

Query type	Det.	ProbSPARQL (ms)				Overhead
	(ms)	$d = 1$	$d = 2$	$d = 4$	$d = 8$	(×)
Retrieval	170.66	213.44	216.32	224.98	253.82	1.25–1.49
Probabilistic filtering	172.16	245.66	246.75	250.03	260.96	1.43–1.52
Uncertainty propagation	31.91	54.31	57.66	67.72	101.24	1.70–3.17
Distribution comparison	–	4339.60	7084.54	15319.34	42768.71	–

than MC-Naive, while avoiding the standalone Det-Bound strategy's stronger bias in returned pairs.

Relation to per-pair speedups. The speedups in Table S.6 are computed over complete-query runtime, which includes graph-pattern evaluation, candidate-pair enumeration, divergence decisions, and result materialization. They are therefore smaller than the per-candidate-pair speedups reported in the main paper, where the divergence decision cost is isolated. For example, Adaptive-Cascade achieves a 7.9× per-pair speedup over MC-Naive on the Hard workload, but a 5.64× complete-query speedup. On the Mixed workload, the corresponding speedups are 24.1× and 11.23×. The difference reflects fixed query-level costs that are not reduced by changing the divergence-decision strategy.

S.8.8 Dimensionality Scaling for GMM Literals

This experiment evaluates the effect of GMM dimensionality on ProbSPARQL execution. We fix the number of mixture components to $K = 3$ and vary the dimension $d \in \{1, 2, 4, 8\}$. The experiment uses the fixed-size S3 benchmark graph with 1,000 generated angle-grinder instances, so that graph size remains unchanged across all dimensionalities. The GMM covariance type is `diag`, which isolates dimensionality effects from the additional cost of full-covariance matrix handling. Component means are sampled in $[0, 10]^d$, and per-dimension variances are sampled in $[0.1, 1.0]$. All operator arguments, including CDF evaluation points and transformation parameters, are chosen to be dimension-compatible for the corresponding GMM dimensionality.

Table S.7 shows that increasing dimensionality has only a limited effect on retrieval and probabilistic filtering, but a stronger effect on uncertainty propagation and distribution comparison. Retrieval is mainly dominated by graph matching, literal parsing, and materialization. Filtering adds dimension-compatible CDF evaluation, while propagation and comparison involve more expensive datatype-specific numerical operations.

Distribution comparison shows the strongest dependence on d, growing by roughly one order of magnitude from $d = 1$ to $d = 8$. This growth reflects the combined cost of higher-dimensional GMM evaluation, per-sample computations in the sampling-based JSD estimator, and fixed query-processing overhead. We

do not isolate these factors in this experiment; the goal is to characterize end-user-visible runtime under increasing dimensionality rather than to provide a microbenchmark of the multivariate Gaussian density kernel.

S.8.9 Evaluation of Datatype Extensibility

We evaluate the polymorphic operator interface using three probabilistic datatype implementations: GMM, Dirichlet, and histogram literals. The experiment uses R2 as the representative distribution-comparison query because it exercises the datatype-specific implementation of `prob:jsd`. On a controlled 500-entity subset, the same query template is evaluated over all three datatype backends and validated against independent reference estimates. Median per-comparison latency is 0.1 ms for histograms, 1.2 ms for Dirichlet distributions, and 3.8 ms for GMMs.

Cross-type comparisons are defined only when both operands can be mapped to a common reference domain. For comparisons involving histograms, the histogram bin partition provides this domain: parametric operands are projected to bin masses, which enables a deterministic-bound pre-filter. For GMM–Dirichlet comparisons, the controlled test cases use an explicit application-level reference-domain mapping and are routed to the sequential decision strategy. If no compatible mapping is configured, the cross-type operator is undefined and expression evaluation yields an error.

Table S.8. Cross-type distribution-comparison evaluation on controlled test cases with $\epsilon = 0.1$ and 10^6-sample reference estimates. Precision, recall, and F1 are reported as percentages; latency is reported as median per-comparison latency in milliseconds.

Combination	Decision route	Prec.	Rec.	F1	Lat.
GMM vs Histogram	Det-Bound + bins	98.2	96.8	97.5	1.8
Dirichlet vs Histogram	Det-Bound + bins	97.1	94.9	96.0	2.3
GMM vs Dirichlet	SPRT	94.8	92.2	93.5	5.5

Table S.8 shows that cross-type comparison remains feasible under explicit reference-domain mappings. Histogram-based combinations are faster because their bin partitions support deterministic pre-filtering, whereas GMM–Dirichlet comparisons require the configured mapping and fall back to SPRT. The results support datatype extensibility at the operator-interface level while making the domain-mapping assumption explicit.

S.9 Algebraic Properties and Proofs

This section records algebraic properties that are summarized in the main paper. We use the set-based solution-mapping notation of the main paper. The same

rewrites extend to bag semantics when multiplicities are inherited from the corresponding ordinary joins and selections.

S.9.1 Theta-Join as Filter-After-Join

Let Θ be a finite set of atomic divergence constraints of the form $\texttt{?x} \sim_{\epsilon,\alpha} \texttt{?y}$. For each atomic constraint

$$\theta = (\texttt{?x} \sim_{\epsilon,\alpha} \texttt{?y}),$$

let F_θ be the corresponding Boolean filter predicate

$$F_\theta = \texttt{prob:divergenceTest}(\texttt{?x}, \texttt{?y}, \epsilon, \alpha).$$

For a finite set $\Theta = \{\theta_1, \ldots, \theta_m\}$, define

$$F_\Theta = F_{\theta_1} \wedge \cdots \wedge F_{\theta_m}.$$

We assume that all variables referenced by F_Θ are in scope after joining the two operands and that the divergence constraints are filter-definable by the corresponding Boolean predicates. Selection keeps exactly those mappings for which the filter evaluates to true; false and error cases are discarded, as in standard SPARQL filter evaluation.

Proposition 1 (Theta-join as filter-after-join). *Let Ω_1 and Ω_2 be sets of solution mappings, and let Θ be a finite set of filter-definable divergence constraints whose variables are in scope over $\Omega_1 \bowtie \Omega_2$. Then*

$$\Omega_1 \bowtie_\Theta \Omega_2 = \sigma_{F_\Theta}(\Omega_1 \bowtie \Omega_2).$$

Thus, the dedicated Θ-join operator does not increase expressive power over an algebra that already contains the corresponding probabilistic filter predicates. Its role is to expose divergence-based matching as an optimizer-visible algebraic operator.

Proof. By the definition of ordinary join, a solution mapping belongs to $\Omega_1 \bowtie \Omega_2$ iff it has the form $\mu_1 \uplus \mu_2$ for some $\mu_1 \in \Omega_1$ and $\mu_2 \in \Omega_2$ such that μ_1 and μ_2 are compatible.

By the definition of Θ-join, the same combined mapping $\mu_1 \uplus \mu_2$ belongs to $\Omega_1 \bowtie_\Theta \Omega_2$ iff, in addition to ordinary mapping compatibility, every atomic divergence constraint in Θ is satisfied by μ_1 and μ_2. For each atomic constraint $\theta \in \Theta$, the corresponding filter predicate F_θ is true under the combined mapping $\mu_1 \uplus \mu_2$ exactly when θ is satisfied. Hence F_Θ is true under $\mu_1 \uplus \mu_2$ exactly when all constraints in Θ are satisfied. Therefore,

$$\mu_1 \uplus \mu_2 \in \Omega_1 \bowtie_\Theta \Omega_2 \quad \text{iff} \quad \mu_1 \uplus \mu_2 \in \sigma_{F_\Theta}(\Omega_1 \bowtie \Omega_2).$$

The two expressions therefore denote the same set of solution mappings.

S.9.2 Projection Conditions

Projection interacts with divergence constraints in the same way it interacts with ordinary SPARQL filters: a filter can only be evaluated after projection if all variables required by the filter remain in scope.

Proposition 2 (Safe projection). *Let W be a set of projected variables. If* $\mathrm{var}(F_\Theta) \subseteq W$*, then*

$$\pi_W\big(\sigma_{F_\Theta}(\Omega)\big) = \sigma_{F_\Theta}\big(\pi_W(\Omega)\big).$$

If $\mathrm{var}(F_\Theta) \nsubseteq W$*, this equivalence need not hold.*

Proof. If $\mathrm{var}(F_\Theta) \subseteq W$, projection preserves every variable needed to evaluate F_Θ. Therefore, for every solution mapping μ, the truth value of F_Θ under μ is the same as its truth value under the projected mapping $\pi_W(\mu)$. Selecting before projection and selecting after projection therefore retain the same projected mappings.

If $\mathrm{var}(F_\Theta) \nsubseteq W$, at least one variable required by the filter is removed by projection. The filter may then evaluate to error because a required variable is unbound, whereas it may have evaluated to true or false before projection. Hence the equivalence can fail.

For Θ-joins, this means that variables used by later divergence constraints must not be projected away before the corresponding compatibility decision has been evaluated.

S.9.3 Reassociation Conditions

For pure inner joins without intervening projection, Θ-joins can be reassociated by rewriting them as ordinary joins followed by conjunctions of filter predicates. The key condition is that each divergence predicate is evaluated only after all variables it references are in scope.

Let $\Omega_1, \Omega_2, \Omega_3$ be solution-mapping sets, and let Θ be a finite set of divergence constraints over variables drawn from the three operands. Let F_Θ be the conjunction of the corresponding filter predicates. Whenever all variables required by F_Θ are in scope after the three-way join,

$$\sigma_{F_\Theta}\big((\Omega_1 \bowtie \Omega_2) \bowtie \Omega_3\big) = \sigma_{F_\Theta}\big(\Omega_1 \bowtie (\Omega_2 \bowtie \Omega_3)\big),$$

because ordinary join is associative and the same conjunction of divergence predicates is evaluated over the same combined mappings.

Proposition 3 (Safe reassociation of local theta-joins). *Let $\Omega_1, \Omega_2, \Omega_3$ be sets of solution mappings such that the variables contributed by the three operands are pairwise disjoint. Let Θ_{12} contain only constraints whose variables come from the first and second operands, and let Θ_{23} contain only constraints whose variables come from the second and third operands. Then*

$$(\Omega_1 \bowtie_{\Theta_{12}} \Omega_2) \bowtie_{\Theta_{23}} \Omega_3 = \Omega_1 \bowtie_{\Theta_{12}} (\Omega_2 \bowtie_{\Theta_{23}} \Omega_3).$$

Proof. By Proposition 1, each local Θ-join can be rewritten as an ordinary join followed by the corresponding filter. Since the operands are pairwise variable-disjoint, ordinary join compatibility introduces no additional cross-operand equality constraints beyond those already represented by the ordinary join structure. The constraint set Θ_{12} depends only on variables from Ω_1 and Ω_2, and Θ_{23} depends only on variables from Ω_2 and Ω_3. Therefore, both sides evaluate the same ordinary three-way join and apply the same two local divergence predicates, only in a different association order. Since ordinary join is associative and both predicates are evaluated once all their variables are in scope, the two sides return the same set of solution mappings.

However, an implementation that evaluates divergence constraints early must respect variable scope. A constraint involving variables from Ω_1 and Ω_3 cannot be evaluated during a join between Ω_1 and Ω_2, because the variable from Ω_3 is not yet bound. Such constraints must be delayed until both operands contributing the referenced variables have been joined.

S.9.4 Interaction with OPTIONAL, UNION, MINUS, and DIFF

The equivalence in Proposition 1 is stated for inner joins. Other SPARQL algebra operators require additional care because probabilistic operators are partial and SPARQL filters have three-valued behavior: true, false, and error.

OPTIONAL. For optional patterns, moving a divergence predicate across an optional boundary can change results when variables used by the predicate may remain unbound. If a distribution-valued variable is introduced only by the optional branch, then evaluating `prob:divergenceTest` outside the optional pattern yields an error on mappings where that branch did not match. This can remove the entire left-side mapping, whereas keeping the predicate inside the optional branch only affects whether the optional bindings are produced. Such rewrites are therefore safe only when all variables referenced by the divergence predicate are guaranteed to be bound at the point where the predicate is evaluated.

UNION. For unions, a filter can be distributed over branches only when the same predicate is evaluated under the same variable bindings and error behavior. In particular,

$$\sigma_F(\Omega_1 \cup \Omega_2) = \sigma_F(\Omega_1) \cup \sigma_F(\Omega_2)$$

under the usual selection semantics. However, an implementation must not push a divergence predicate into a branch or below an operator where the variables needed by the predicate are not yet in scope. Branches that do not bind the required variables will make the predicate evaluate to error, and this behavior must be preserved by the rewrite.

MINUS and DIFF. For difference-like operators, moving divergence predicates across the operator can change which mappings are removed. Predicates that depend only on variables from the left-hand side can be treated as ordinary

selections on the preserved mappings. Predicates that depend on variables from the right-hand side, or on variables from both sides, must not be moved across the difference boundary without an explicit side condition, because the predicate then affects the existence of removing mappings. This is especially important when variables are unbound, probabilistic literals are malformed, or datatype/domain compatibility checks produce errors.

Error propagation. Malformed probabilistic literals, incompatible domains, dimensionality mismatches, or unsupported datatype/operator combinations make probabilistic operator evaluation undefined. In filter contexts, such undefined evaluations produce errors and are handled according to standard SPARQL filter error propagation. Consequently, algebraic rewrites involving probabilistic predicates are valid only when they preserve both variable scope and error behavior.

S.10 Implementation Details and Reproducibility

S.10.1 Apache Jena ARQ Extension

The prototype extends Apache Jena ARQ with probabilistic datatype interpretation, probabilistic expression evaluation, and divergence-join execution. Probabilistic RDF literals are interpreted as internal value objects during operator evaluation. Datatype-specific backends implement distribution evaluation, transformation, uncertainty propagation, divergence estimation, and serialization back to RDF literals when query results contain distribution-valued outputs.

The extended query processor can be used through the ARQ execution layer and, for endpoint-based experiments, through an Apache Jena Fuseki deployment. This allows probabilistic filters and divergence-based joins to be evaluated inside the SPARQL engine rather than by exporting distribution literals to an external post-processing script.

S.10.2 Datatype Registry

The probabilistic datatype layer is organized as a registry. Each registered datatype provides:

- a parser from lexical forms to internal probabilistic value objects;
- a serializer from internal value objects back to RDF literals;
- a value representation for the corresponding distribution family;
- implementations of supported distribution-valued and deterministic-valued operators;
- divergence or distance routines where comparison operators are supported.

This design separates graph-pattern evaluation from datatype-specific probabilistic computation. Additional probabilistic datatypes can therefore be added by registering a new backend, without changing the surrounding SPARQL graph-pattern evaluation machinery.

S.10.3 Artifact Structure

The reproducibility package is organized as follows:

- `src/`: ProbSPARQL implementation, Apache Jena ARQ extensions, datatype registry, and datatype-specific operator backends.
- `data/`: generated circular-factory benchmark graphs and configuration files for dataset generation.
- `queries/`: ProbSPARQL query workloads for R1–R4, dimensionality-scaling experiments, end-to-end divergence-join evaluation, and cross-type extensibility experiments.
- `scripts/`: scripts for dataset generation, benchmark execution, and result aggregation.
- `results/`: raw benchmark outputs and summarized experimental results for the runs reported in the paper.
- `README.md`: artifact setup instructions, software requirements, execution workflow, and description of the expected outputs.

The benchmark scripts fix random seeds for generated distributions and query workloads where randomness is involved. This makes the generated RDF graphs and candidate-pair workloads reproducible from the artifact configuration, while the benchmark scripts recompute the reported aggregate metrics under the local execution environment.

S.10.4 Reproduction Workflow

The experiments can be reproduced using the following workflow.

1. **Build the prototype.** Install the required Java environment and build the ProbSPARQL extension together with the datatype backends.
2. **Generate or load benchmark data.** Use the provided dataset-generation scripts, or load the generated RDF benchmark graphs from the artifact package. The generated graphs follow the circular-factory schema described in Section S.7.
3. **Start the query engine.** Run the extended Apache Jena ARQ processor directly or start the configured Apache Jena Fuseki endpoint with the ProbSPARQL extension and registered probabilistic datatypes.
4. **Execute query workloads.** Run the R1–R4 query workloads from Section S.6 and, where applicable, the extended evaluation workloads for divergence-decision strategies, end-to-end divergence joins, dimensionality scaling, and cross-type datatype extensibility from Section S.8.
5. **Aggregate results.** Use the provided aggregation scripts to compute the metrics used by each experiment, including median latency, standard deviation, precision, recall, F1 score, and mean absolute error where applicable. Unless otherwise stated, latency values are aggregated as the median of 10 measured runs after 3 warm-up iterations.